Context workflows

Give each task the source files it needs

Context selection

Choose sources a reviewer can explain:

- repository instructions;
- relevant source files and tests;
- architecture or API documents;
- task notes.

Context packet exercise

```
Task: add validation to an API endpoint

Packet:
- route file
- corresponding tests
- API contract
- repository conventions
```

Ask learners why each source is needed and what is excluded.

Evaluate the result

- Does it follow the selected repository files?
- Is the scope limited to the task?
- Are assumptions listed for review?
- Do relevant tests pass?

Agenda and time plan

Time	Topic
0:00	Why context fails
0:08	Context layers
0:18	Spaces
0:30	Instructions and prompt files
0:42	Keeping context fresh
0:50	Before/after demo
0:58	Lab handoff

The context problem

Copilot can't apply information it can't see. Extra context adds noise.

Common failures:

- the relevant contract is absent;
- stale documentation conflicts with source code;
- broad context hides the key constraint;
- local preferences are mistaken for team standards.

A practical context hierarchy

```
Task intent and acceptance criteria
                ↓
Repository instructions and prompt workflow
                ↓
Curated Space: code, docs, issues, specifications
                ↓
Current file and explicit references
                ↓
Memory, when available
```

Specific task constraints override generic guidance. Clarify conflicts.

Copilot Spaces

A Space is a selected set of sources for a recurring topic or workflow.

Good candidates:

- source repositories and key paths;
- architecture and API documentation;
- issues and specifications;
- team guidance for a domain.

Curate for a purpose

```
Space: Storefront API
├── API specification
├── product route and model
├── architecture note
├── error-handling conventions
└── repository instructions
```

Exclude unrelated services, production data, secrets, abandoned proposals, and documents nobody keeps current.

Trainer cue: Ask learners what they would remove first if the answer became noisy.

Sharing Spaces

Before sharing:

1. Name the audience and task.
2. Add purpose-specific instructions.
3. Test with a known question and compare against the source files.
4. Remove or replace stale sources.

Repository-level instructions

Create `.github/copilot-instructions.md` for project-wide guidance:

```
# Repository guidance

- Use TypeScript strict mode.
- Validate request input at the route boundary.
- Keep business logic in `src/services/`.
- Run `npm test` before proposing completion.
- Do not add dependencies without reviewer approval.
```

Write rules a reviewer can check. Avoid vague goals such as "produce high-quality code."

Reusable prompt files

Place team prompts under `.github/prompts/`:

```
---
description: Review an API change against its contract
---

Read the selected route, service, tests, and API contract.
List mismatches before suggesting changes.
Do not edit files outside the selected feature.
End with the focused validation command.
```

Inspect what the CLI loaded

```
/env
/instructions
```

Find the copied stale CommonJS note and its route-file scope.
Compare it with `package.json` and the current ES-module route.

Correct only that training note, restart, and inspect again.
Leave personal, managed, and security instructions alone.

Discovery is not proof of application. Check the proposed code too.

VS Code instruction settings

```
{
  "github.copilot.chat.codeGeneration.useInstructionFiles": true
}
```

Settings change between IDE versions. Check the setting in your learners' version.

Copilot Memory

Memory can retain knowledge across sessions, where available. Ask three questions:

1. **Source:** Where did this fact come from?
2. **Scope:** Does it apply to this user, repository, or task?
3. **Freshness:** How will it be corrected when the code changes?

Never store secrets.

Demo: without grounded context

Prompt:

```
Add a low-stock report endpoint.
```

Discuss:

- Which endpoint?
- Which inputs and error schema?
- Where is validation normally performed?
- Which tests prove the behavior?

Don't score the response by confidence or length.

Demo: with a context packet

```
Using the Storefront API specification, product route, product model, architecture
note, and repository instructions, propose the smallest low-stock endpoint change.

First list acceptance criteria and assumptions. Do not add dependencies.
End with two checks to run and cite the files used.
```

Compare assumptions, source references, and the checks each response proposes.

Concrete code target

```
function parseThreshold(value) {
  if (value === undefined) return 10;
  if (!/^\d+$/.test(value)) {
    throw new Error("threshold must be a non-negative integer");
  }
  return Number(value);
}
```

Ask where the static report route must appear relative to `/:id`.

Keep context fresh

Trigger	Action
API contract changes	Update Space sources and prompts
Repository convention changes	Review instruction files
Output contradicts source	Remove stale sources and retest

Lab handoff

You will:

1. keep one Storefront API for the full lab;
2. select the smallest source set;
3. add repository instructions and a low-stock endpoint;
4. compare one logging task across context levels;
5. store and test three reusable prompt files.

Remember

1. Relevant context beats a large packet.
2. Select Space sources for a named purpose and audience.
3. Instructions state project-wide rules. Prompt files define reusable tasks.
4. Memory needs a source, a scope, and a way to correct it.

Questions

Which source would you remove from a context packet, and why?

- Which source is authoritative?
- What must stay out of the Space?